

MEDIATEK

MT8676 Yocto Camera Sensor Driver JSON Arch User Manual

Version: 1.0
Release date: 2025-06-12

Use of this document and any information contained therein is subject to the terms and conditions set forth in Exhibit 1. This document is subject to change without notice.

Version History

Version	Date	Author	Description
1.0	2025-06-12	Jianmin Zhou	Official release

Table of Contents

Version History 2

Table of Contents 3

1 Camera Driver JSON Arch 4

 1.1 Overview 4

 1.2 Configuration/Customization Guideline 4

 1.2.1 How to Add a New Sensor 4

 1.2.2 How to Add a New Tuning File 26

Exhibit 1 Terms and Conditions 28

1 Camera Driver JSON Arch

1.1 Overview

This chapter primarily discusses the sensor porting method for the MT8676 camera post decoupling Refactor, along with the debugging approach and considerations for related issues.

For the design architecture and principles of camera refactor, please refer to the following document: *MT8678 JSON Design EN.pptx*.

1.2 Configuration/Customization Guideline

This section primarily discusses the porting guide for the camera refactor decoupling architecture.

1.2.1 How to Add a New Sensor

JSON conversion only supports YUV sensors.

1.2.1.1 Porting File List

Yocto:

- Kernel Space
 - \src\kernel\linux\v6.1_mt8676\co_device_modules\arch\arm64\boot\dts\mediatek\cust_mt8676_camera_v4l2.dtsi
 - \src\kernel\modules\mt8676\camera\imgsensor\src-v4l2\\$(sensor_name).json
- User Space
 - \src\multimedia\camera-hal\mt8676\mtkcam-core\external\firmware\sensor\\$(sensor_name).bin
 - \src\multimedia\camera-hal\mt8676\mtkcam-core\external\firmware\camera\metadata\\$(sensor_name)\\$(sensor_name).json
 - \src\multimedia\camera-hal\mt8676\hardware\mtkcam-core\external\firmware\camera\tuning\\$(sensor_name)_tuning_param.json
- Config Files
 - \src\meta\meta-mediatek-mt8676\recipes-multimedia\mtkcam-mt8676\mtkcam-mt8676.bb

1.2.1.2 Add Sensor Firmware Files to the Kernel

Yocto:

- Kernel Space
 - \src\kernel\modules\mt8676\camera\imgsensor\src-v4l2
- User Space
 - \src\multimedia\camera-hal\mt8676\mtkcam-core\external\firmware\sensor\\$(sensor_name).bin

$\$(sensor_name).json$ file is generated by a dedicated tool: **Camera Firmware JSON Generator**. This tool is a web page interface where developers can fill in the required form content according to the prompts on the page, and finally click save to generate $\$(sensor_name).json$ file.

The meaning of the parameters in the web tool form can be found in the document *MT8678 JSON Design EN.ppt*.

Camera Firmware Json Generator - For Automotive

MEDIATEK

載入 JSON

Click “載入 JSON” to load existing sensor JSON files for modification.

Form

```

=== camera_module ===
* name:
YourSensor_mipi_yuv

* compatible_dts_mask:
0x1

* module_id:
0x52

* module_id_is_unique:
YES

i2c_addr_table
0x52
0xff

```

Json

```

1 {
2   "camera_module": {
3     "name": "YourSensor_mipi_yuv",
4     "compatible_dts_mask": "0x1",
5     "module_id": "0x52",
6     "module_id_is_unique": 1,
7     "module_id_virtual": "0xff",
8     "i2c_addr_table": [
9       "0x52",
10      "0xff",
11      "0xff",
12      "0xff",
13      "0xff"
14    ],
15    "reg_addr_sensor_id": [
16      "0x00",
17      "0x00",
18      "0x00"
19    ],
20    "i2c_transfer_data_type": 0,
21    "reg_cnt_for_id": 1
22  },
23  "serdes": {
24    "serializer": {
25      "vendor": "maxim",
26      "name": "max92xxx",
27      "index": 0,
28      "linked_deser_idx": 0,
29      "i2c_transfer_data_type": 0,
30      "vc_id": 0
31    }
32  }
33 }

```

The $\$(sensor_name).json$ file that is generated needs to be saved to the `\src\kernel\modules\mt8676\camera\imgsensor\src-v4l2\` directory.

> merged > src > kernel > modules > mt8676 > camera > imgsensor > src-v4l2 >

Name	Date modified	Type	Size
max96712a3_mipi_yuv	4/7/2025 7:01 PM	JSON File	14 KB
max96712a2_mipi_yuv	4/7/2025 7:01 PM	JSON File	14 KB
max96712a1_mipi_yuv	4/7/2025 7:01 PM	JSON File	14 KB
max96712a0_mipi_yuv	4/7/2025 7:01 PM	JSON File	14 KB
max96712a_mipi_yuv	4/7/2025 7:01 PM	JSON File	33 KB
max96712_mipi_yuv_ss220at	4/7/2025 7:01 PM	JSON File	33 KB

Execute the command “`python3 gen_sensor_firmware.py ./sensor_name.json`” in the `src-v4l2/` directory.

```

gen_sensor_firmware.py
python3 gen_sensor_firmware.py ./max96712_mipi_yuv_ss220at.json

```

After the successful execution of the script, a file named `sensor_name.bin` is generated in the `src-v4l2/` directory.

<< merged > src > kernel > modules > mt8676 > camera > imgsensor > src-v4l2

Name	Date modified	Type	Size
max96712_mipi_yuv.bin	5/10/2025 2:48 PM	BIN File	3 KB

The generated $\$(sensor_name).bin$ binary file can be pushed to the `/vendor/firmware/sensor/` directory on the development board using `adb`. Subsequently, the development board can be restarted.

```
_64_bsp:/vendor/firmware/sensor # ls -l
total 8
-rw-r--r-- 1 root root 1751 2023-12-28 05:40 sc20at_mipi_yuv.bin
-rw-r--r-- 1 root root 1751 2023-12-28 05:40 sc21at_mipi_yuv.bin
-rw-r--r-- 1 root root 1751 2023-12-28 05:40 sc22at_mipi_yuv.bin
-rw-r--r-- 1 root root 1751 2023-12-28 05:41 sc23at_mipi_yuv.bin
```

Note: By pushing the `firmware.bin` file to the `/vendor/firmware/sensor/` directory on the development board using `adb`, firmware debugging can be facilitated without the need to rebuild the driver source code repeatedly.

After confirming that the firmware does not require any further modifications, save the JSON file. Subsequently, during the next full build process, all firmware files will be compiled into the image. Following the complete burning process, the development board system will include all `firmware.bin` files.

1.2.1.3 Modify dts Files

Yocto:

```
\src\kernel\linux\v6.1_mt8676\co_device_modules\arch\arm64\boot\dts\mediatek\cust_mt8676_camera_v4l2.dtsi
```

By analyzing the hardware schematic, the current sensor's connection to a specific I2C can be identified. Then, sensor configuration should be added under the corresponding I2C, as illustrated below:

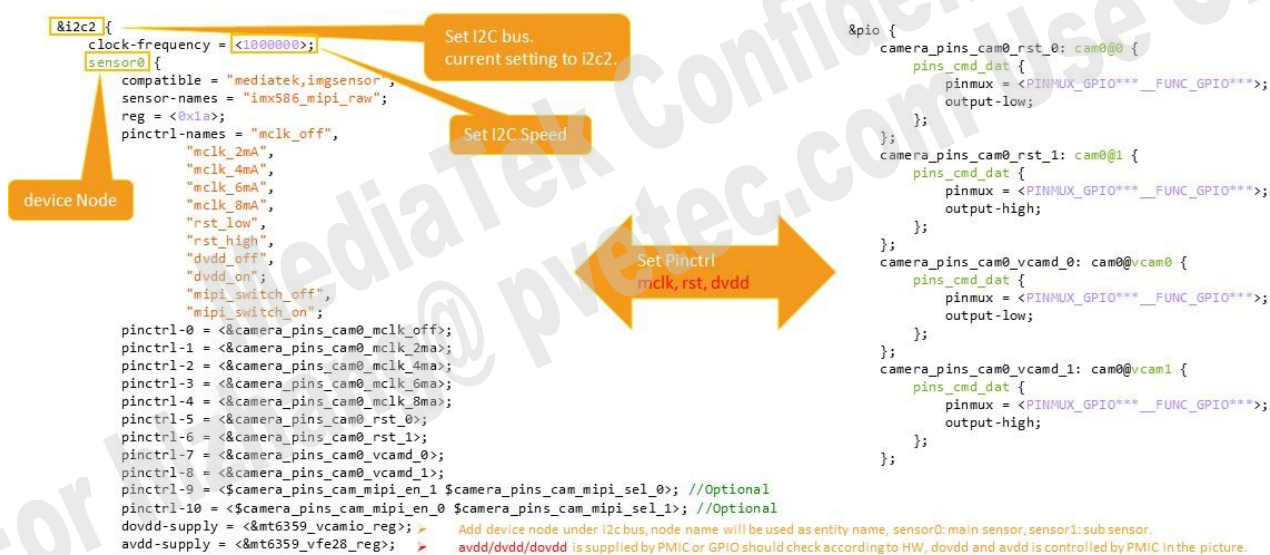
- Added Sensor0

Note: If the sensor needs to be changed in the legacy method, then a new attribute needs to be added under the sensor node: **mediatek, legacy-search;**

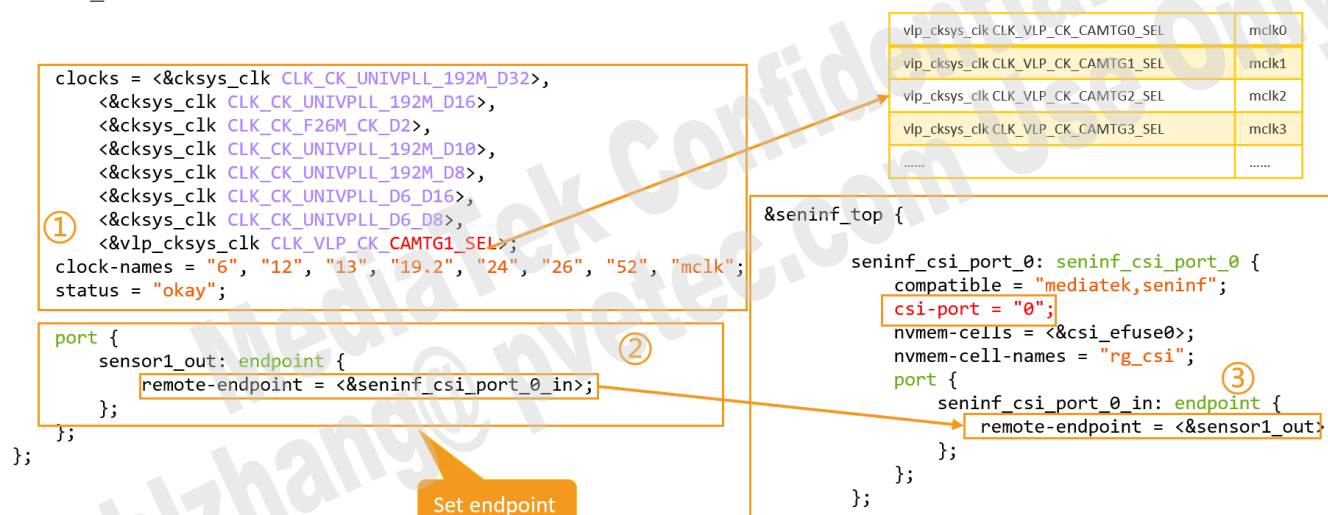
```
sensor1: sensor1@52 {
    » mediatek, legacy-search;
    » compatible = "mediatek, imgsensor";
    » sensor-names = "max96712_mipi_yuv";
    » reg = <0x52>;

    » #thermal-sensor-cells = <0>;
    » pdn-supply = <&gpio47_regulator>;

    » clocks = <&vlp_cksys_clk CLK CK_UNIVPLL_192M_D32>,
    »          <&vlp_cksys_clk CLK CK_UNIVPLL_192M_D16>,
    »          <&vlp_cksys_clk CLK CK_F26M_CK_D2>;
```



Concerning the correlation between seninf and sensor, please refer to the diagram below. The value of `csi-port` in `seninf_top` represents the actual hardware-connected `csi-port`.



- `CLK_VLP_CK_CAMTG1_SEL` is mclk source, currently set to `mclk1`.
- The setting of csi port represents which CSI the current sensor is connected to. For example, 0, 1, 2, 3, 0A, 0B, 1A, 1B etc.
- The port setting Check seninf's csi-port property, currently `sensor1` connect to `CSI0`.

The part corresponding to the configuration in dts is as follows. It is necessary to refer to the sensor's specification sheet for configuring the power-on sequence.

1. Add power sequence info

- {power_type, voltage, delay_time in ms}
- Power on sequence from top to bottom
- Power off sequence from bottom to top
- mclk set frequency(24M) and drive current(4mA)

2. Add subdrv entry

- Naming rule: \$(folder-name)_entry
- Macro will generate sensor-list automatically

➤ Rename a copy according to the existing sensor driver source code, and the part of the register and sensor info should be confirmed with the vendor.

common/imx586_mipi_raw/imx586mipiraw_Sensor.c

```
static struct subdrv_pw_seq_entry pw_seq[] = {
    {HW_ID_MCLK, 24, 0},
    {HW_ID_RST, 0, 0},
    {HW_ID_AVDD, 2800000, 0},
    {HW_ID_AFVDD, 2800000, 0},
    {HW_ID_DVDD, 1100000, 0},
    {HW_ID_DOVDD, 1800000, 1},
    {HW_ID_MCLK_DRIVING_CURRENT, 4, 1},
    {HW_ID_RST, 1, 3},
};

const struct subdrv_entry imx586_mipi_raw_entry = {
    .name = "imx586_mipi_raw",
    .id = IMX586_SENSOR_ID,
    .pw_seq = pw_seq,
    .pw_seq_cnt = ARRAY_SIZE(pw_seq),
    .ops = &ops,
};
```

Additional note: For the fixed regulator, para1 and para2 need to be configured the same as in the dts, with identical values.

The format of pw_seq in the most recent version of the code max96712mipiuv_sensor.c is as follows:

```
969 static struct subdrv_pw_seq_entry pw_seq[] = {
970     {HW_ID_PDN, 0, 20},
971     {HW_ID_PDN, 1800000, 20},
972 };
```

1.2.1.4 Add New Meta Files

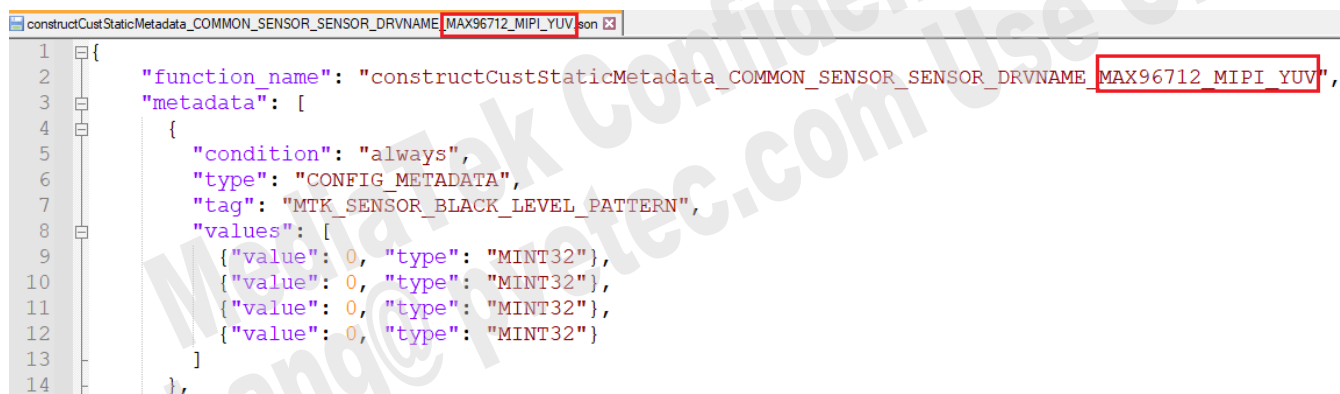
Copy a metadata template folder (\src\multimedia\camera-hal\mt8676\mtkcam-core\external\firmware\metadata\max96712_mipi_yuv), and change the folder name to the sensor name currently being ported (e.g., xxxx_mipi_yuv).

Modify the sensor name in the names of all JSON files under the xxxx_mipi_yuv folder and also in these files to the name of the sensor currently being ported. Ensure that the letter casing matches the format used in the template file.

Take max96712_mipi_yuv as an example. The folder names in all JSON files containing max96712_mipi_yuv or MAX96712_MIPI_YUV need to be changed to the name of the sensor currently being ported.

merged > src > multimedia > camera-hal > mt8676 > mtkcam-core > external > firmware > metadata > max96712_mipi_yuv

Name	Date modified	Type
constructCustRequestMetadata_DEVICE_SENSOR_DRVNAME_MAX96712_MIPI_YUV	4/7/2025 7:01 PM	JSON File
constructCustStaticMetadata_COMMON_SENSOR_SENSOR_DRVNAME_MAX96712_MIPI_YUV	4/7/2025 7:01 PM	JSON File
constructCustStaticMetadata_PLATFORM_AVAILABLE_KEYS_SENSOR_DRVNAME_MAX96712_MIPI_YUV	4/7/2025 7:01 PM	JSON File
constructCustStaticMetadata_PLATFORM_CUSTOM_SENSOR_DRVNAME_MAX96712_MIPI_YUV	4/7/2025 7:01 PM	JSON File
constructCustStaticMetadata_PLATFORM_LENS_SENSOR_DRVNAME_MAX96712_MIPI_YUV	4/7/2025 7:01 PM	JSON File
constructCustStaticMetadata_PLATFORM_MOD_OVERWRITE_SENSOR_DRVNAME_MAX96712_MIPI_YUV	4/7/2025 7:01 PM	JSON File
constructCustStaticMetadata_PLATFORM_MODULE_SENSOR_DRVNAME_MAX96712_MIPI_YUV	4/7/2025 7:01 PM	JSON File
constructCustStaticMetadata_PLATFORM_PRO_OVERWRITE_SENSOR_DRVNAME_MAX96712_MIPI_YUV	4/7/2025 7:01 PM	JSON File
constructCustStaticMetadata_PLATFORM_PROJECT_SENSOR_DRVNAME_MAX96712_MIPI_YUV	4/7/2025 7:01 PM	JSON File
constructCustStaticMetadata_PLATFORM_VENDOR_SENSOR_DRVNAME_MAX96712_MIPI_YUV	4/7/2025 7:01 PM	JSON File
constructCustStaticMetadata_PROJECT_V1_PRO_OVERWRITE_SENSOR_DRVNAME_MAX96712_MIPI_YUV	4/7/2025 7:01 PM	JSON File



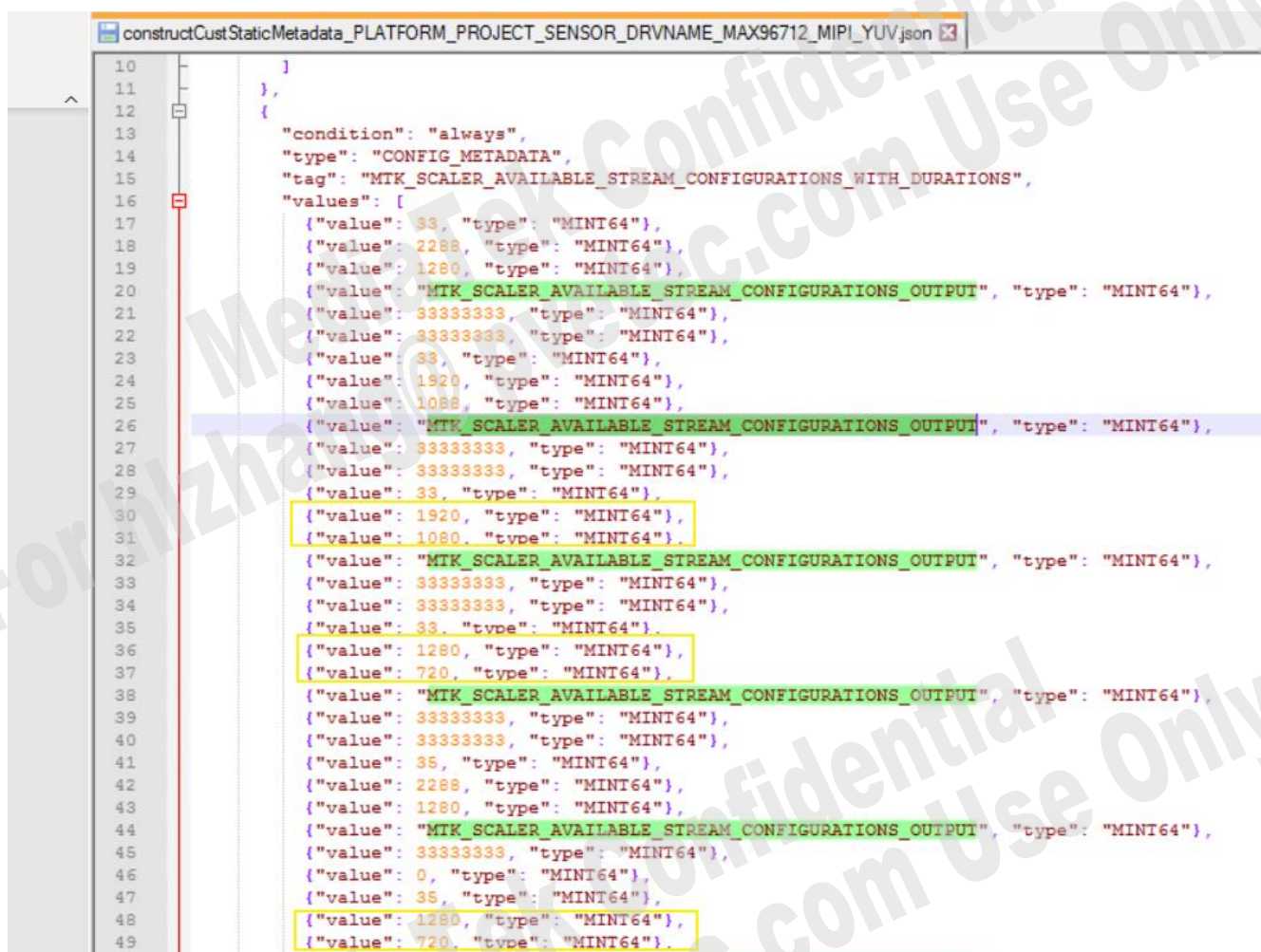
```

1  {
2      "function_name": "constructCustStaticMetadata_COMMON_SENSOR_SENSOR_DRVNAME_MAX96712_MIPI_YUV",
3      "metadata": [
4          {
5              "condition": "always",
6              "type": "CONFIG_METADATA",
7              "tag": "MTK_SENSOR_BLACK_LEVEL_PATTERN",
8              "values": [
9                  {"value": 0, "type": "MINT32"},
10                 {"value": 0, "type": "MINT32"},
11                 {"value": 0, "type": "MINT32"},
12                 {"value": 0, "type": "MINT32"}
13             ]
14         }
15     ]
16 }

```

Note:

- A sensor has several different types of JSON files (all with the suffix _SENSOR_DRVNAME_xxx.json), and each JSON file needs to be modified in the manner described above.
- If it is a raw or another type of sensor, you need to copy the corresponding sensor type's JSON and modify it based on that.
- If you need to modify the resolution, you can refer to the following JSON and change it to the new resolution.



```

10  }
11  },
12  {
13      "condition": "always",
14      "type": "CONFIG_METADATA",
15      "tag": "MTK_SCALER_AVAILABLE_STREAM_CONFIGURATIONS_WITH_DURATIONS",
16      "values": [
17          {"value": 33, "type": "MINT64"},
18          {"value": 2288, "type": "MINT64"},
19          {"value": 1280, "type": "MINT64"},
20          {"value": "MTK_SCALER_AVAILABLE_STREAM_CONFIGURATIONS_OUTPUT", "type": "MINT64"},
21          {"value": 33333333, "type": "MINT64"},
22          {"value": 33333333, "type": "MINT64"},
23          {"value": 33, "type": "MINT64"},
24          {"value": 1920, "type": "MINT64"},
25          {"value": 1080, "type": "MINT64"},
26          {"value": "MTK_SCALER_AVAILABLE_STREAM_CONFIGURATIONS_OUTPUT", "type": "MINT64"},
27          {"value": 33333333, "type": "MINT64"},
28          {"value": 33333333, "type": "MINT64"},
29          {"value": 33, "type": "MINT64"},
30          {"value": 1920, "type": "MINT64"},
31          {"value": 1080, "type": "MINT64"},
32          {"value": "MTK_SCALER_AVAILABLE_STREAM_CONFIGURATIONS_OUTPUT", "type": "MINT64"},
33          {"value": 33333333, "type": "MINT64"},
34          {"value": 33333333, "type": "MINT64"},
35          {"value": 33, "type": "MINT64"},
36          {"value": 1280, "type": "MINT64"},
37          {"value": 720, "type": "MINT64"},
38          {"value": "MTK_SCALER_AVAILABLE_STREAM_CONFIGURATIONS_OUTPUT", "type": "MINT64"},
39          {"value": 33333333, "type": "MINT64"},
40          {"value": 33333333, "type": "MINT64"},
41          {"value": 35, "type": "MINT64"},
42          {"value": 2288, "type": "MINT64"},
43          {"value": 1280, "type": "MINT64"},
44          {"value": "MTK_SCALER_AVAILABLE_STREAM_CONFIGURATIONS_OUTPUT", "type": "MINT64"},
45          {"value": 33333333, "type": "MINT64"},
46          {"value": 0, "type": "MINT64"},
47          {"value": 35, "type": "MINT64"},
48          {"value": 1280, "type": "MINT64"},
49          {"value": 720, "type": "MINT64"}
50      ]
51  }
52 }

```

After the modification is completed, the xxxx_mipi_yuv/ folder should be pushed to the /vendor/firmware/camera/metadata/ folder on the development board using adb push. The system should then be rebooted.

After confirming that no more modifications are needed for the metadata, place the respective metadata file in the \src\multimedia\camera-hal\mt8676\mtkcam-core\external\firmware\metadata\max96712_mipi_yuv directory. Subsequently, execute the full build command to build all firmware files into the image. Following complete burning, all metadata files will be included in the development board system.

Note: If errors similar to the following occur during the full build process, it indicates differences in native basic tags between different versions. During version migration, some tags will be synchronized to mtk_metadata_tag.h, and therefore a script will be needed to update MapStringToEnum.cpp.

```
mtkcam-core/external/MapStringToEnum.cpp:738:41: error: use of undeclared identifier
'MTK_MFNR_FEA                               TURE_STORE_BSS2META'; did you mean
'MTK_MFNR_FEATURE_START'?
{"MTK_MFNR_FEATURE_STORE_BSS2META", MTK_MFNR_FEATURE_STORE_BSS2META},
^~~~~~
```

Update the script execution method: Go to the mtkcam-core/external/ directory, and execute python MapStringToEnum.py . .

The full build command should be executed again.

1.2.1.5 Configure Sensor Format

Modification should be made in the web interface **Camera Firmware JSON Generator**.

Configure UYVY or YUYV according to the format output by the bridge:

sensor_output_dataformat:

SENSOR_OUTPUT_FORMAT_UYVY

Locate all instances where sensor_output_dataformat is configured and change them to the actual format.

1.2.1.6 Configure Mode Structure

Modification should be made in the web interface **Camera Firmware JSON Generator**.

The relevant parameters of the setting are to be confirmed with the vendor and filled into the structure.

Pclk->pixel clock, Unit: Hz

Linlength->HTS

Framelength->VTS

Linlength × framelength = pclk

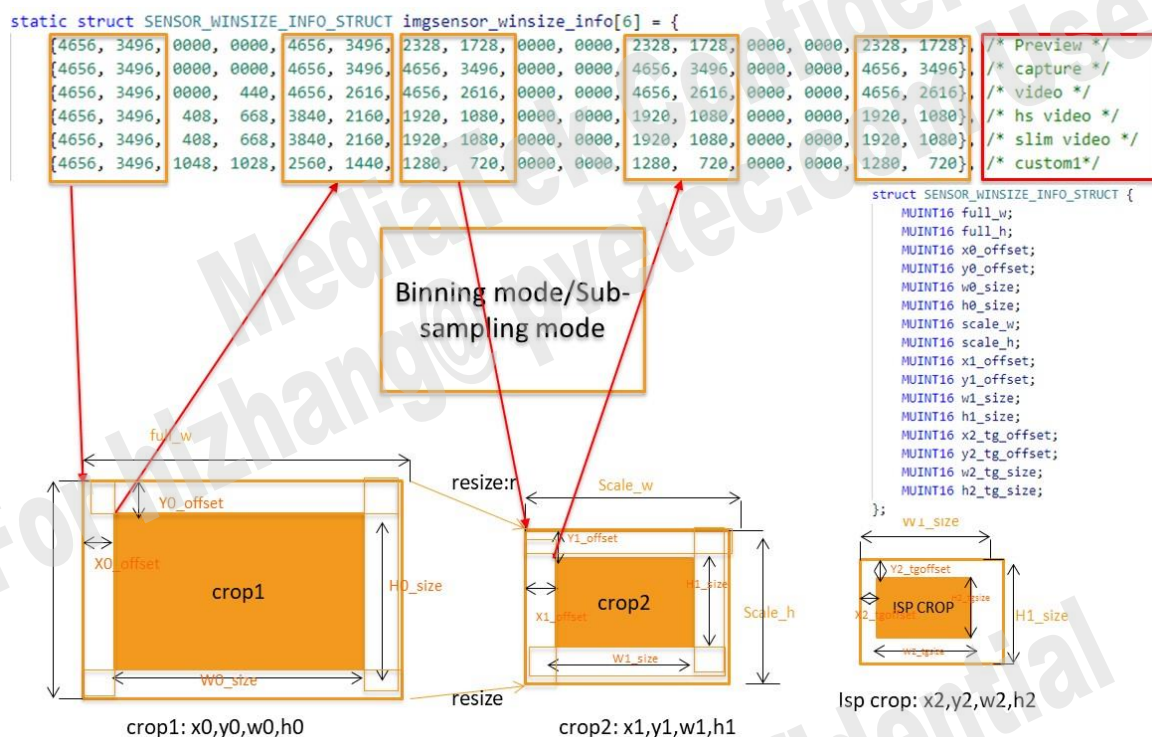
Grabwindow_width -> camera size width (This value is not available on MT8676 and will be filled in the solution section later).

Grabwindow_height -> camera size height (This value is not available on MT8676 and will be filled in the solution section later).

Mipi_pixel_rate-> mipi datarate × lane_number/bitdepth, YUV bitdepth = 16



The following diagram borrows an explanation of winsize info from a previous version to describe the meanings of each field.



Limitation: Full size (4:3) or full size (16:9) width and height need 4x alignment.

The size (width and height) of the sensor output should match the size of `Grabwindow_width/Grabwindow_height`.

Grab window request:

1. The width must be a multiple of 16 and the height must be a multiple of 4.

- The viewing angles are consistent at the same ratio; the ratios of 4:3 and 16:9 require that the window output by the sensor ensures that the horizontal direction is consistent with the full size.
- The grab window setting recommendations are consistent with the window output by the sensor. In special cases: $\text{grabwindow width}(\text{height}) \leq \text{sensor output width}(\text{height}) - \text{startx}(\text{y})$

1.2.1.7 Configure static_ctx

Modification should be made in the web interface **Camera Firmware JSON Generator**.

- The `sensor_id` field below is filled with the corresponding sensor ID.
- The `i2c_add_table` below generally only needs to be configured with 0x52, which is the actual address of the deserializer.
- Configure the resolution; this is the value of grabwindow mentioned earlier.
- Configure `mipi_lane_num`, which represents the number of MIPI lanes here.

```

111  "subdrv_static_ctx": {
112    "i2c_burst_write_support": 1,
113    "i2c_transfer_data_type": 0,
114    "resolution": [
115      1920,
116      1080
117    ],
118    "mirror": 3,
119    "mclk": 24,
120    "isp_driving_current": 1,
121    "sensor_interface_type": 1,
122    "mipi_sensor_type": 0,
123    "mipi_lane_num": {
124      "value": 0
125    },
126    "sensor_output_dataformat": 4,
127    "frame_length_max": 65532,
128    "ae_effective_frame": 2,
129    "frame_time_delay_frame": 3,
130    "start_exposure_offset": 500000,
131    "seamless_switch_support": 0,
132    "temperature_support": 0,
133    "reg_addr_stream": 256,
134    "reg_addr_mirror_flip": 257,
135    "chk_s_off_sta": 1,
136    "chk_s_off_sta": 0
  }

```

- `cam_type` is filled according to the type of module scene. If it is a single sensor, you can fill in `MTK_SENSOR_TYPE_SINGLE`, which has the following values to choose:

```

523  enum SPM_CAM_TYPE_ENUM {
524    SENSOR_TYPE_SINGLE = 0,
525    SENSOR_TYPE_COMB = 1,
526    SENSOR_TYPE_MULTI = 2,
527    SENSOR_TYPE_NON_COMB = 3,
528    SENSOR_TYPE_COMB_AVM = 4,
529    SENSOR_TYPE_NON_COMB_AVM = 5,
530    SENSOR_TYPE_MULTI_ASYNC = 6,
531    SENSOR_TYPE_EXTCAM = 7,
532    SENSOR_TYPE_NONE
533  };

```

- `SENSOR_TYPE_COMB_AVM` represents the combined frame AVM.
- `SENSOR_TYPE_MULTI` represents multicam.
- `SENSOR_TYPE_MULTI_ASYNC` Indicates a multicam camera with independent VC channels.
- `SENSOR_TYPE_NON_COMB_AVM` represents non-combined AVM.

- `sensor_output_dataformat` represents the format of the data output by the module, filled in the actual format.

sensor_output_dataformat:

SENSOR_OUTPUT_FORMAT_UYVY

SENSOR_OUTPUT_FORMAT_UYVY

SENSOR_OUTPUT_FORMAT_VYUY

SENSOR_OUTPUT_FORMAT_YUYV

SENSOR_OUTPUT_FORMAT_VYU

Regarding the configuration of multicam group info, refer to the following configuration in the JSON web tool:

multicam_group_inf

master_dts_idx:

11

+

-

vc_num:

4

+

-

slave_group_info

* slave_dts_idx:

7

+

-

* vc_id:

0

+

-

* link_status:

0

+

-

* slave_dts_idx:

8

+

-

* vc_id:

1

+

-

* link_status:

0

+

-

* slave_dts_idx:

9

+

-

* vc_id:

2

+

-

* link_status:

0

+

-

* slave_dts_idx:

10

+

-

* vc_id:

3

+

-

* link_status:

0

+

-

multicam_slave_num:

The format in the final JSON file will include sensor ID and VC information from one master sensor and four slave sensors in their respective DTS files.

MEDIATEK

MediaTek Proprietary and Confidential. © 2025 MediaTek Inc. All rights reserved. Unauthorized reproduction or disclosure of this document, in whole or in part, is strictly prohibited.

13

```

959 "multicam_group_inf": {
960   "master_dts_idx": 11,
961   "vc_num": 4,
962   "slave_group_info": [
963     {
964       "slave_dts_idx": 7,
965       "vc_id": 0,
966       "link_status": 0
967     },
968     {
969       "slave_dts_idx": 8,
970       "vc_id": 1,
971       "link_status": 0
972     },
973     {
974       "slave_dts_idx": 9,
975       "vc_id": 2,
976       "link_status": 0
977     },
978     {
979       "slave_dts_idx": 10,
980       "vc_id": 3,
981       "link_status": 0
982     }
983   ]
984 }
985 },

```

Regarding the configuration of `multicam_group_info`, you can refer to the following configuration:

You need to fill in the number of VC channels, and you need to fill in four channels. If it is a multicam with only two or three drivers, you also need to fill in the information of 4 channels. For channels that are not used, just fill in 0 for `vc_data_type`.

```

16 "multicam_group_dt_inf": {
17   "vc_num": 4,
18   "sensor_group_dt_info": [
19     {
20       "vc_id": 0,
21       "vc_data_type": 30
22     },
23     {
24       "vc_id": 1,
25       "vc_data_type": 30
26     },
27     {
28       "vc_id": 2,
29       "vc_data_type": 30
30     },
31     {
32       "vc_id": 3,
33       "vc_data_type": 30
34     }
35   ]
36 }

```

1.2.1.8 Configure VC Information

Modification should be made in the web interface **Camera Firmware JSON Generator**.

A single VC can be configured as follows:

frame_desc

* channel:

0

* data_type:

0

* dt_remap_to_type:

0

```

51 },
52 "frame_desc": [
53 {
54   "channel": 0,
55   "data_type": 0,
56   "dt_remap_to_type": 0,
57   "hsize": 1,
58   "vsize": 1,
59   "user_data_desc": 0,
60   "fs_seq": 0
61 }
62 ],
63 "mode_struct": [
64 {
65   "hdr_mode": 0,
66   "raw_cnt": 1,
67   "exp_cnt": 1,

```

If there are multiple VCs, primarily modify the channel and user_data_desc.

1.2.1.9 Configure to Get the Sensor ID

Modification should be made in the web interface **Camera Firmware JSON Generator**.

=== camera_module ===

* name:

YourSensor_mipi_yuv

* compatible_dts_mask:

0x1

* module_id:

0x52

* module_id_is_unique:

YES

```

1 {
2   "camera_module": {
3     "name": "YourSensor_mipi_yuv",
4     "compatible_dts_mask": "0x1",
5     "module_id": "0x52",
6     "module_id_is_unique": 1,
7     "module_id_virtual": "0xff",
8     "i2c_addr_table": [
9       "0x52",
10      "0xff",
11      "0xff",
12      "0xff",
13      "0xff"
14    ],
15    "reg_addr_sensor_id": [
16      "0x00",
17      "0x00",
18      "0x00"
19    ],

```

1.2.1.10 Configure Sensor Init

Modification should be made in the web interface **Camera Firmware JSON Generator**.

The primary task involves configuring the module's settings, which necessitates debugging and generation by both the customer and the vendor.

=== reg_init_settings ===

init_serdes_reg_info

+

init_cmos_reg_info

+

```

38   },
39 },
40 "reg_init_settings": {
41   "init_serdes_reg_info": [
42   ],
43   "init_cmos_reg_info": [
44   ],
45 },
46 "reg_streaming_settings": {
47   "stream_on_reg_info": [
48   ],
49   "stream_off_reg_info": [
50   ],
51 },
52 "frame_desc": [
53 {
54   "channel": 0,

```

1.2.1.11 Configure get_sensor_usage

Modification should be made in the web interface **Camera Firmware JSON Generator**.

```

subdrv_entry
* sensor_usage:
MTK_SENSOR_USAGE_SINGLE
163 },
164 "pw_seq": [
165 ],
166 "subdrv_entry": {
167   "sensor_usage": 0
168 },

```

Its values can be selected as follows: COMB represents combined frame AVM; MUTI represents multicam; NONCOMB represents non-combined frame AVM; MUTICAM_SYNC represents a multi cam camera where the slave is independent and does not affect each other, and for a single sensor configuration, it should be set as SINGLE.

```

42 enum mtk_sensor_usage {
43     MTK_SENSOR_USAGE_SINGLE,
44     MTK_SENSOR_USAGE_COMB,
45     MTK_SENSOR_USAGE_MUTI,
46     MTK_SENSOR_USAGE_NONCOMB,
47     MTK_SENSOR_USAGE_MUTI_ASYNC,
48 };

```

1.2.1.12 Configure Streaming Control

Modify in the web interface **Camera Firmware JSON Generator**.

All initialization settings should be placed within the `sensor_init()` function. The functions for enabling and disabling MIPI should be placed in the `streaming_control()` function.

```

=== reg_streaming_settings ===
stream_on_reg_info
+
stream_off_reg_info
+
43 "init_cmos_reg_info": [
44 ],
45 },
46 "reg_streaming_settings": {
47   "stream_on_reg_info": [
48   ],
49   "stream_off_reg_info": [
50   ]
51 },
52 "frame_desc": [
53 {
54   "channel": 0,
55   "data_type": 0,
56   "dt_remap_to_type": 0,
57   "hsize": 1,
58   "vsize": 1,
59   "user_data_desc": 0,

```

1.2.1.13 subdrv_static_ctx Structure Member Analysis

Member	Meaning
sensor_id	Sensor ID define in kd_imgsensor.h.
reg_addr_sensor_id	Sensor register address where sensor ID is read. Up to 3 bytes.
i2c_addr_table	I2C write ID, end in 0xff, 4 for maximum. Ex. i2c_addr_table = {0x20, 0x6e, 0xff},.
eeeprom_info	The address of eeeprom_info_struct. If there is no eeeprom device, set to 0.
eeeprom_num	Array size of the eeeprom_info_struct. If there is no eeeprom device, set to 0.

Member	Meaning
resolution	Full pixel size of sensor output.
mirror	Set IMAGE_HV_MIRROR if the sensor orientation is 180 degrees difference from module orientation.
mclk	Overwrite MCLK frequency (MHz)
isp_driving_current	Overwrite MCLK driving current (mA)
sensor_interface_type	Data output interface type. Set default MIPI.
mipi_sensor_type	C-PHY or D-PHY
mipi_lane_num	How many lanes/trios of MIPI-PHY
ob_pedestal	OB offset. General is 64.
sensor_output_dataformat	Bayer order, 4-cell order and SW/HW remo.
ana_gain_def	Default analog gain. Set 4x. 1024base.
ana_gain_min	Minimum analog gain. 1024base.
ana_gain_max	Maximum analog gain. 1024base.
ana_gain_type	Sony: type 0; OV: type 1; Samsung: type 2; Hynix: type 3; GC: type 4
ana_gain_step	Minimum valid step of analog gain. 1024base.
ana_gain_table	Use valid analog gain table. Remove unsuitable value which is not linearity. 1024base.
ana_gain_table_size	Size of analog gain table
min_gain_iso	Set minimum ISO 100.
exposure_def	Default exposure line. Set 0x3D0.
exposure_min	Minimum exposure line
exposure_max	Maximum exposure line
exposure_step	Minimum valid step of exposure line
exposure_margin	Maximum margin of exposure line
frame_length_max	Maximum framelength
ae_effective_frame	AE effective frame
frame_time_delay_frame	The frame "frame length" setting takes effect. Sony sensor is filled in 3; other sensors are filled in 2.
start_exposure_offset	Parameter tuned in CTS sensor fusion test.
pdaf_type	Reference enum IMGSENSOR_PDAF_SUPPORT_TYPE_ENUM
hdr_type	Reference enum IMGSENSOR_HDR_SUPPORT_TYPE_ENUM
seamless_switch_support	If sensor supports seamless switch, set 1 to enable seamless switch function.
temperature_support	Set to 1 if sensor supports temperature sensor readout.
g_temp	Implement get temperature function.
g_gain2reg	Implement analog gain to register mapping rule.
s_gph	Implement set enable/disable group hold function.
s_cali	Implement the function to write data from eeprom_info_struct to sensor (ex: QSC, cross-talk, ...)

Member	Meaning
Member	Meaning
reg_addr_stream	Sensor register address where streaming on/off is controlled.
reg_addr_mirror_flip	Sensor register address where mirror/flip on/off is controlled. No use to set to 0.
reg_addr_exposure	Sensor register address where exposure line is set. Up to 3 channels of exposure. Up to 3 bytes.
long_exposure_support	Set to 1 if sensor supports long exposure left shift function.
reg_addr_exposure_lshift	Sensor register address where long exposure left shift is set.
reg_addr_ana_gain	Sensor register address where analog gain is set. Up to 3 channels of exposure. Up to 3 bytes.
reg_addr_frame_length	Sensor register address where framelength is set.
reg_addr_temp_en	If "temperature_support" is set to 1. Sensor register address where temperature sensor on/off is controlled.
reg_addr_temp_read	If "temperature_support" is set to 1. Sensor register address where temperature output is read.
reg_addr_auto_extend	For Sony sensor, sensor register address where auto extend function enable/disable is controlled.
reg_addr_frame_count	For Samsung sensor, sensor register address where frame count is read to determine if sensor stream is off or not.
reg_addr_fast_mode	For Sony sensor, sensor register address where fast mode on/off is controlled.
init_setting_table	The address of sensor initial setting table
init_setting_len	Array size of the sensor initial setting table
mode	The address of subdrv_mode_struct
sensor_mode_num	Array size of the subdrv_mode_struct
list	The address of customized feature control list
list_len	Array size of the customized feature control list
checksum_value	The calculated value when Test Pattern output, for Camera Auto Test

1.2.1.14 Sensor Mode Setting

Subdrv_mode_struct mode_struct[] structure array is used to store data configurations for different Sensor Modes. The following example using "preview" illustrates some fields that customers need to modify. For a YUV sensor, the sensor mode and preview are generally identical. However, for a raw sensor, there may be differences based on specific requirements for configuration.

mode_struct

hdr_mode:

0

raw_cnt:

1

exp_cnt:

1

pclk:

linelength:

framelength:

```

61 | }
62 | ],
63 | "mode_struct": [
64 | {
65 |   "hdr_mode": 0,
66 |   "raw_cnt": 1,
67 |   "exp_cnt": 1,
68 |   "pclk": undefined,
69 |   "linelength": undefined,
70 |   "framelength": undefined,
71 |   "max_framerate": undefined,
72 |   "mipi_pixel_rate": undefined,
73 |   "readout_length": 0,
74 |   "read_margin": 10,
75 |   "framelength_step": 2,
76 |   "coarse_integ_step": 2,
77 |   "pdaf_cap": 1,
78 |   "ae_binning_ratio": 1465,
79 |   "fine_integ_line": 826,
80 |   "delay_frame": 3,
81 |   "dpc_enabled": 1,
82 |   "csi_param": {
83 |     "dphy_trail": undefined,
84 |     "dphy_data_settle": undefined,
85 |     "dphy_clk_settle": undefined,
86 |     "cphy_settle": undefined,
87 |     "legacy_phy": undefined,
88 |     "not fixed trail settle": undefin

```

imgsensor_winsize_info

* full_w:

* full_h:

* x0_offset:

* y0_offset:

* w0_size:

```

94 | },
95 | "imgsensor_winsize_info": {
96 |   "full_w": undefined,
97 |   "full_h": undefined,
98 |   "x0_offset": undefined,
99 |   "y0_offset": undefined,
100 |   "w0_size": undefined,
101 |   "h0_size": undefined,
102 |   "scale_w": undefined,
103 |   "scale_h": undefined,
104 |   "x1_offset": undefined,
105 |   "y1_offset": undefined,
106 |   "w1_size": undefined,
107 |   "h1_size": undefined,
108 |   "x2_tg_offset": undefined,
109 |   "y2_tg_offset": undefined,
110 |   "w2_tg_size": undefined,
111 |   "h2_tg_size": undefined
112 | },
113 | "sensor_output_dataformat": 4,
114 | "ana_gain_min": 1024,
115 | "ana_gain_max": 65536,
116 | "dig_gain_min": 1024,

```

sensor_output_dataformat:

SENSOR_OUTPUT_FORMAT_UYVY

```

111 |   h2_tg_size: undefined
112 | },
113 | "sensor_output_dataformat": 4,
114 | "ana_gain_min": 1024,
115 | "ana_gain_max": 65536,
116 | "dig gain min": 1024,

```

csi_param

dphy_trail:

dphy_data_settle:

dphy_clk_settle:

cphy_settle:

```

82 | "csi_param": {
83 |   "dphy_trail": undefined,
84 |   "dphy_data_settle": undefined,
85 |   "dphy_clk_settle": undefined,
86 |   "cphy_settle": undefined,
87 |   "legacy_phy": undefined,
88 |   "not fixed trail settle": undefined,
89 |   "dphy_csi2_resync_dmy_cycle": undefined,
90 |   "not fixed dphy_settle": undefined,
91 |   "dphy_init_deskew_support": undefined,
92 |   "clk_lane_no_initial_flow": 0,
93 |   "initial_skew": undefined
94 | },
95 | "imgsensor_winsize_info": {
96 |   "full_w": undefined,
97 |   "full_h": undefined,
98 |   "x0_offset": undefined,
99 |   "y0_offset": undefined

```

The following is the explanation in .c code.

```

00344: static struct subdrv_mode_struct |mode_struct[] = {
00345: {
00346:     .frame_desc = frame_desc_prev,
00347:     .num_entries = ARRAY_SIZE(frame_desc_prev),
00348:     .mode_setting_table = PARAM_UNDEFINED,
00349:     .mode_setting_len = PARAM_UNDEFINED,
00350:     .seamless_switch_group = PARAM_UNDEFINED,
00351:     .seamless_switch_mode_setting_table = PARAM_UNDEFINED,
00352:     .seamless_switch_mode_setting_len = PARAM_UNDEFINED,
00353:     .hdr_mode = HDR_NONE,
00354:     .raw_cnt = 1,
00355:     .exp_cnt = 1,
00356:     .pclk = 500000000,
00357:     .linelength = 8096,
00358:     .framelength = 1400,
00359:     .max_framerate = 300,
00360:     .mipi_pixel_rate = 500000000,
00361:     .readout_length = 0,
00362:     .read_margin = 10,
00363:     .framelength_step = 2,
00364:     .coarse_integ_step = 2,
00365:     .sensor_output_dataformat = SENSOR_OUTPUT_FORMAT_UYVY,
00366:     .multi_exposure_shutter_range[IMGSSENSOR_EXPOSURE_LE].min = 8,
00367:     .imgsensor_winsize_info = {
00368:         .full_w = SUPERFRAME_WIDTH,
00369:         .full_h = SUPERFRAME_HEIGHT,
00370:         .x0_offset = 0,
00371:         .y0_offset = 0,
00372:         .w0_size = SUPERFRAME_WIDTH,
00373:         .h0_size = SUPERFRAME_HEIGHT,
00374:         .scale_w = SUPERFRAME_WIDTH,
00375:         .scale_h = SUPERFRAME_HEIGHT,
00376:         .x1_offset = 0,
00377:         .y1_offset = 0,
00378:         .w1_size = SUPERFRAME_WIDTH,
00379:         .h1_size = SUPERFRAME_HEIGHT,
00380:         .x2_tg_offset = 0,
00381:         .y2_tg_offset = 0,
00382:         .w2_tg_size = SUPERFRAME_WIDTH,
00383:         .h2_tg_size = SUPERFRAME_HEIGHT,
00384:     },
00385:     .pdaif_cap = TRUE,
00386:     .imgsensor_pd_info = NULL,
00387:     .ae_binning_ratio = 1465,
00388:     .fine_integ_line = 826,
00389:     .delay_frame = 3,
00390:     .csi_param = {
00391:         .legacy_phy = 0,
00392:         .not_fixed_trail_settle = 1,
00393:         .not_fixed_dphy_settle = 1,
00394:         .dphy_trail = 0x4,
00395:         .dphy_data_settle = 0x20,
00396:         .cphy_settle = 13,
00397:         .clk_lane_no_initial_flow = 0,
00398:     },
00399:     .dpc_enabled = true, /* reg 0xb06 */
00400: }, /* preview */

```

Configure Pclk, framelength, framerate, mipi picel rate

Configure format

Configure sinsize info

Configure hs trail & settle

1.2.1.15 Driver Functions

1.2.1.15.1 Driver Operation Function List

The list of driver functions is as follows:

```

1073 static struct subdrv_ops ops = {
1074     .get_id = get_imgsensor_id,
1075     .init_ctx = init_ctx,
1076     .open = open,
1077     .get_info = common_get_info,
1078     .get_resolution = common_get_resolution,
1079     .control = common_control,
1080     .feature_control = common_feature_control,
1081     .close = common_close,
1082     .get_frame_desc = common_get_frame_desc,
1083     .get_temp = common_get_temp,
1084     .get_csi_param = common_get_csi_param,
1085     .update_sof_cnt = common_update_sof_cnt,
1086     .parse_ebd_line = common_parse_ebd_line,
1087     .get_sensor_usage = get_sensor_usage,
1088 };

```

In the code for the imgsensord starting with the adapter name, it is often called to the driver code through the subdrv_call method, which is the function list above, for example:

```
ret = subdrv_call(ctx, get_id, &sensor_id);
```

The call above will be called to the get_sensor_id in the function list above.

There are many functions in the list above that start with “common”. This section utilizes a common processing flow. It does not need to be specifically implemented in the sensor driver. Further elaboration will not be provided below.

The following is a description of the get_imgsensor_id and open functions that your company may change.

1.2.1.15.2 get_imgsensor_id Function

During startup, the search sensor process retrieves the ID by utilizing the get_imgsensor_id function. If the retrieval of the ID is successful, the UI will display the camera APP’s icon.

The primary purpose of the following function is to acquire the sensor ID and determine whether there is a link to the module.

```

751 static int get_imgsensor_id(struct subdrv_ctx *ctx, u32 *sensor_id)
752 {
753     // u32 deser_id, ret = 0;
754     // dts_id represents sensor 0/1/2/3 index
755     u32 compatible_dts_mask = 0xFFFFFFFF;
756     struct autocam_sector_data *sec_data = NULL;
757     struct adaptor_ctx *adp_ctx = container_of(ctx, struct adaptor_ctx, subctx);
758     sec_data = ctx->s_sec_data;
759     if (!sec_data) {
760         LOG_ERR("sec_data is NULL, pls check!");
761         return ERROR_DRIVER_INIT_FAIL;
762     }
763
764     compatible_dts_mask = sec_data->autocam_module_inf->compatible_dts_mask;
765     if ((compatible_dts_mask & (1 << adp_ctx->dts_idx)) == 0) {
766         LOG_ERR("firmware[%s] is not compatible with DTS sensor[%d]",
767             sec_data->file_name, adp_ctx->dts_idx);
768         return -ERROR_SENSOR_CONNECT_FAIL;
769     }
770     // ret = get_deserializer_id(ctx, &deser_id);
771     // if (ret < 0) {
772     //     LOG_ERR("SensorDevIdx: %d, can not find deserializer: %s", 1 << adp_ctx->dts_idx, sec_data->autocam_serdes_inf->deser_inf->name);
773     //     return ret;
774     // }
775     LOG_INF("SensorDevIdx: %d, expect cam module_id: 0x%x", 1 << adp_ctx->dts_idx, sec_data->autocam_module_inf->module_id);
776     get_cam_module_id(ctx, sensor_id);
777     // if sensor_id read by i2c is not the same as the one from firmware data, return connect failure.
778     if (*sensor_id != sec_data->autocam_module_inf->module_id) {
779         *sensor_id = 0xFFFFFFFF;
780         return -ERROR_SENSOR_CONNECT_FAIL;
781     }
782     LOG_INF("sensor_id = 0x%x, matched with expect module_id", *sensor_id);
783     return 0;
784 }

```

1.2.1.15.3 Open Function

This function will be called every time when the camera is accessed.

Read `sensor_id` to verify if the I2C communication is functioning properly.

The `sensor_init` function is called to initialize some variables in the `ctx` structure.

```

int open(struct subdrv_ctx *ctx)
{
    u8 retry = 2;
    u32 sensor_id = 0;
    u32 scenario_id = 0;

    LOG_INF("%s:\n", __func__);
    get_imgsensor_id(ctx, &sensor_id);
    autocam_init(ctx);
    retry = 3;
    do {
        if (Check_Sensor_Signal(ctx)) {
            LOG_ERR("Sensor Not Connect");
            mdelay(5);
            retry--;
        } else
            break;
    } while (retry > 0);

    if (retry == 0) {
        LOG_ERR("OPEN FAIL!Sensor Not Connect");
        return -ERROR_SENSOR_CONNECT_FAIL;
    }

    /* move camera init to initctx(), void max9286 init too much time */

    spin_lock(&autocam_drv_lock);
    ctx->current_scenario_id = scenario_id;
    ctx->pclk = ctx->s_ctx.mode[scenario_id].pclk;
    ctx->line_length = ctx->s_ctx.mode[scenario_id].linelength;
    ctx->frame_length = ctx->s_ctx.mode[scenario_id].framelength;
    ctx->current_fps = ctx->pclk / ctx->line_length * 10 / ctx->frame_length;
    ctx->readout_length = ctx->s_ctx.mode[scenario_id].readout_length;
    ctx->read_margin = ctx->s_ctx.mode[scenario_id].read_margin;
    ctx->min_frame_length = ctx->frame_length;
    ctx->autoflicker_en = FALSE;
    ctx->test_pattern = 0;
    ctx->ihdr_mode = 0;
    ctx->pdaf_mode = 0;
    ctx->hdr_mode = 0;
    ctx->extend_frame_length_en = 0;
    ctx->fast_mode_on = 0;
    ctx->sof_cnt = 0;
    ctx->ref_sof_cnt = 0;
    ctx->is_streaming = 0;
    spin_unlock(&autocam_drv_lock);

    return ERROR_NONE;
}

```

Note:**1. Explanation about resolution**

Due to different scenarios, the size of the image provided to the user space and the size of VC actually received by seninf are different.

So there will be two resolutions in the driver, and customers can use the driver of max96712isx, a non-combined frame AVM, as an example.

The following macros should be searched for to compare configurations.

```

00026: #define VC 0
00027:
00028: #define SUPERFRAME_WIDTH 7680
00029: #define SUPERFRAME_HEIGHT 1280
00030:
00031: #define FRAME_WIDTH 1920
00032: #define FRAME_HEIGHT 1280
00033:
00034: #define MAX96712_ID_REG 0x00U
00035: #define SC22AT_I2C_ADDR I 0x60
00036: #define MAX96717_I2C_ADDR 0x80
00037: #define MAX96712_SENSOR_ID 0x52
00038:

```

This is the maximum resolution seen by the upper layer.
If it is a non-combined frame AVM, the width here will be 7680
single VC channel resolution

2. Regarding the part of GPIO power supply in DTS, you can also choose the regulator packaging method.

The usage is as follows.

```

109 >     gpio46_regulator: gpio-46 {
110 >         compatible = "regulator-fixed";
111 >         regulator-name = "gpio46_regulator";
112 >         regulator-min-microvolt = <1800000>;
113 >         regulator-max-microvolt = <1800000>;
114 >         gpio = <&gpio 46 0x0>;
115 >         enable-active-high;
116 >         status = "okay";
117 >     };

686 >     sensor0: sensor0@52 {
687 >         compatible = "mediatek,ingsensor";
688 >         sensor-names = "max96712isx_mipi_yuv";
689 >         reg = <0x52>;
690 >
691 >         #thermal-sensor-cells = <0>;
692 >         pdn-supply = <&gpio46_regulator>;
693 >
694 >         clocks = <&vlp_cksys_clk CLK_CK_UNIVPLL_192M_D32>,
695 >                 <&vlp_cksys_clk CLK_CK_UNIVPLL_192M_D16>,
696 >                 <&vlp_cksys_clk CLK_CK_F26M_CK_D2>,
697 >                 <&vlp_cksys_clk CLK_CK_UNIVPLL_192M_D10>,
698 >                 <&vlp_cksys_clk CLK_CK_UNIVPLL_192M_D8>,
699 >                 <&vlp_cksys_clk CLK_CK_UNIVPLL_D6_D16>,
700 >                 <&vlp_cksys_clk CLK_CK_UNIVPLL_D6_D8>,
701 >                 <&vlp_cksys_clk CLK_VLP_CK_CAMT08_SEL>;
702 >         clock-names = "6", "12", "13", "19.2", "24", "26", "52", "mclk";
703 >
704 >         status = "okay";
705 >
706 >         port {
707 >             sensor0_out: endpoint {
708 >                 remote-endpoint = <&seninf_csi_port_4_in>;
709 >             };
710 >         };
711 >     };

```

3. UT test command

```
C:\Users>adb shell sentest_v4l2 //just search sensor, not streaming
```

```

[show_Sensors]sensorNum 2
[show_Sensors]name:SENSOR_DRVNAME_OV05A20_MIPI_RAW type:0
[show_Sensors]index:0, SensorDevIdx:1
[show_Sensors]name:SENSOR_DRVNAME_IMX576_MIPI_RAW type:0
[show_Sensors]index:1, SensorDevIdx:2
[main]Param: 1 <sensorDev> <scenario> <fps>
[main]<sensorDev> : main(1), Sub(2), Main2(4), sub2(8), Main3(16)
[main]<scenario> : Pre(0), Cap(1), VD(2), slim1(3), slim2(4)

```

The output shows that sensorNum is not 0 and display the specific name of the sensor, indicating the search sensor is successful. Then test the output of the sensor.

```
C:\Users> adb shell sentest_v4l2 1 0 //search sensor, then open main (1) sensor
preview (0)
```

```
[test_SensorInterface]name:SENSOR_DRVNAME_MAX96712ISX_MIPI_YUV type:1
[test_SensorInterface]index:0, SensorDevIdx:1
[test_SensorInterface]scenario(0), binning type = 1465
[test_SensorInterface]scenario(0), rawFmtType = 0
[test_SensorInterface]scenario(0), OB_pedestal = 64
[test_SensorInterface]scenario(0), sensorFormatOrder = 4
[test_SensorInterface]scenario(0), seamless_targets_count = 0
[test_SensorInterface]Sensor index:0, scenarioId:0
[test_SensorInterface]crop w:1280, h:720
[test_SensorInterface]HDR_Support:0
[test_SensorInterface]scenario(0), RGBW_Support = 0
[test_SensorInterface]scenario(0), PDAF_Support = 0, dpc(1), pdc(0)
[test_SensorInterface]scenario(0), Def_MirrorFlip = 3
[test_SensorInterface]scenario(0), sensorFormatCellType = 0
[test_SensorInterface]SENSOR_CMD_GET_SENSOR_CROP_WIN_INFO ret 0
[test_SensorInterface]SENSOR_CMD_GET_SENSOR_RGBW_OUTPUT_MODE rgbw mode = 0
[test_SensorInterface]ana gain min/max = 1024/65536
[test_SensorInterface]dig gain min/max = 1024/16384
[test_SensorInterface]dig gain step = 4
[test_SensorInterface]SENSOR_CMD_GET_OFFSET_TO_START_OF_EXPOSURE exposure_offset = 500000
[test_SensorInterface]pHalSensor->powerOn start
[test_CID][test_g_volatile_ctrl] scenario (by set format)
[test_CID][V4L2_CID_VBLANK] ret:0 VB:680
[test_CID][V4L2_CID_HBLANK] ret:0 HB:4434
[test_CID][V4L2_CID_MTK_SENSOR_PIXEL_RATE] ret:0 pixel rate:200000000
[test_CID][V4L2_CID_MTK_STAGGER_INFO] ret:0 count:0 order:
[test_CID][V4L2_CID_MAX_EXP_TIME]ret 0 exposure 0, max_exp_time 0
[test_CID][V4L2_CID_MAX_EXP_TIME]ret 0 exposure 1, max_exp_time 0
[test_CID][V4L2_CID_MAX_EXP_TIME]ret 0 exposure 2, max_exp_time 0
[streamon]sensor: idx=0 devpath=/dev/v4l-subdev26 drvname=max96712isx_mipi_yuv
[streamon]seninf: devpath=/dev/v4l-subdev4 cid=0x98f901 en=1
[test_SensorInterface]main/rolling_shutter=20571428

[test_SensorInterface]main/exp_margin=1371

[test_SensorInterface]main/atv-Continuous=1

[test_SensorInterface]main/atv-BypassSensorScenario=0

[test_SensorInterface]main/min_shutter=24 shutter_step=2

[test_SensorInterface]IgInfo[0] = 0
[test_SensorInterface]pixelMode[0] = 0
[test_SensorInterface]mipi rate = 200000000
[test_SensorInterface]wait enter key to end sensorIndex:0...
```

4. The command to check seninf status is as follows. Once the results are found, they can be sent to MediaTek for confirmation.

```
255|auto8676p1_64_bsp:/ # cat /sys/devices/platform/soc/1a00e000.seninf-top/status
clk-fmeter-isp(687576) clk-fmeter-csi0(26000) clk-fmeter-csi1(0) clk-fmeter-csi2(0) clk-fmeter-csi3(312000) clk-fmeter-csi4(0) clk-fmeter-csi5(0)

[seninf-8] port 3 intf 6 test 0 cphy 0 lanes 0

[seninf-7] port 3 intf 6 test 0 cphy 0 lanes 0
source sensor7 flags 0x4

[seninf-6] port 3 intf 6 test 0 cphy 0 lanes 4
source sensor6 flags 0x5
csi_rx_mac_csi_irq_stat 0x003f0324, seninf_irq_stat 0x00000000
csi2_line_frame_num 0x00000000
csi2_packet_status 0x12c0012b
csi2_packet_cnt_status 0x000004fb
rx-ana_settle_ck 0x00 dt 0x1a
rx-ana_trail_en 0 param 0x00
CSIRX_MAC_CSI2_RESYNC_MERGE_CTRL 0x00000106 CDPHY_RX_ANA_SETTING_1 0x04340244 DPHY_RX_SPARE0 0x000000f1
data_not_enough_cnt : <0>
err_lane_resync_cnt : <0>
crc_err_cnt : <0>
ecc_err_double_cnt : <0>
ecc_err_corrected_cnt : <0>
after_write_clear_csi_irq 0x0 seninf_irq_stat 0x00000000
[0] vc 0x0 dt 0x2b mux 1 cam 0
mux[1] en 1 src 6 irq_stat 0xc
fifo_overrun_cnt : <0>
cam[0] en 1 src 8 exp 0x8700f00 res 0x8700f00 err 0x0 irq_stat 0x100
size_err_cnt : <0>
--- mux meter ---
width 3840 height 2160
h_valid 0, h_blank 0
v_valid 0, v_blank 0
mipi_pixel_rate 0
v_blank 0 us
h_blank 0 us
line_time 0 us

[seninf-2] port 2 intf 4 test 0 cphy 0 lanes 0

[seninf-1] port 1 intf 2 test 0 cphy 0 lanes 0
auto8676p1_64_bsp:/ #
```

5. Access max96712 directly via I2C commands

The following is the value read from register 0x108.

```
echo 0x8a0 0x84> /sys/devices/platform/soc/13b30000.i2c/i2c-8/8-0052/debug_i2c_ops
D:\Debug>adb shell
auto8676p1_64_bsp:/ # echo 0x52 0x108 > /sys/devices/platform/11cc0000.i2c/i2c-8/8-0052/debug_i2c_ops
auto8676p1_64_bsp:/ # cat /sys/devices/platform/11cc0000.i2c/i2c-8/8-0052/debug_i2c_ops
max96712_mipi_yuv i2c read 0x00000108 = 0x00000062
auto8676p1_64_bsp:/ #
```

The following is 0x8a0 register written as 0x84.

```
echo 0x52 0x8a0 0x84> /sys/devices/platform/11cc0000.i2c/i2c-8/8-0052/debug_i2c_ops
```

6. The command to enable sensor driver log

```
echo 1 > /sys/module/imgsensor/parameters/sensor_debug
```

7. If the serial port log printing stops, open the UART command:

```
adb shell setprop persist.vendor.uartconsole.enable 1
```

1.2.2 How to Add a New Tuning File

Design principles reference: *MT8678 JSON Design EN.pptx*

Add method: When porting a new sensor, in the tuning section, simply add a `${SENSOR_DRIVER_NAME}_tuning_param.json` file.

Note:

- If you need to build into the image, place the JSON file in the `\src\multimedia\camera-hal\mt8676\mtkcam-core\external\firmware\camera\tuning` directory
- If you need to add a sensor's tuning JSON file separately, use adb push to push the tuning JSON file to the `vendor/firmware/camera/tuning/` directory

`${SENSOR_DRIVER_NAME}_tuning_param.json`: Copy an existing template and modify the name of `sensor_driver_name`.

Exhibit 1 Terms and Conditions

Your access to and use of this document and the information contained herein (collectively this “Document”) is subject to your (including the corporation or other legal entity you represent, collectively “You”) acceptance of the terms and conditions set forth below (“T&C”). By using, accessing or downloading this Document, You are accepting the T&C and agree to be bound by the T&C. If You don’t agree to the T&C, You may not use this Document and shall immediately destroy any copy thereof.

This Document contains information that is confidential and proprietary to MediaTek Inc. and/or its affiliates (collectively “MediaTek”) or its licensors and is provided solely for Your internal use with MediaTek’s chipset(s) described in this Document and shall not be used for any other purposes (including but not limited to identifying or providing evidence to support any potential patent infringement claim against MediaTek or any of MediaTek’s suppliers and/or direct or indirect customers). Unauthorized use or disclosure of the information contained herein is prohibited. You agree to indemnify MediaTek for any loss or damages suffered by MediaTek for Your unauthorized use or disclosure of this Document, in whole or in part.

MediaTek and its licensors retain titles and all ownership rights in and to this Document and no license (express or implied, by estoppels or otherwise) to any intellectual propriety rights is granted hereunder. This Document is subject to change without further notification. MediaTek does not assume any responsibility arising out of or in connection with any use of, or reliance on, this Document, and specifically disclaims any and all liability, including, without limitation, consequential or incidental damages.

THIS DOCUMENT AND ANY OTHER MATERIALS OR TECHNICAL SUPPORT PROVIDED BY MEDIATEK IN CONNECTION WITH THIS DOCUMENT, IF ANY, ARE PROVIDED “AS IS” WITHOUT WARRANTY OF ANY KIND, WHETHER EXPRESS, IMPLIED, STATUTORY, OR OTHERWISE. MEDIATEK SPECIFICALLY DISCLAIMS ALL WARRANTIES OF MERCHANTABILITY, NON-INFRINGEMENT, FITNESS FOR A PARTICULAR PURPOSE, COMPLETENESS OR ACCURACY AND ALL WARRANTIES ARISING OUT OF TRADE USAGE OR OUT OF A COURSE OF DEALING OR COURSE OF PERFORMANCE. MEDIATEK SHALL NOT BE RESPONSIBLE FOR ANY MEDIATEK DELIVERABLES MADE TO MEET YOUR SPECIFICATIONS OR TO CONFORM TO A PARTICULAR STANDARD OR OPEN FORUM.

Without limiting the generality of the foregoing, MediaTek makes no warranty, representation or guarantee regarding the suitability of its products for any particular purpose, nor does MediaTek assume any liability arising out of the application or use of any product, circuit or software. You agree that You are solely responsible for the designing, validating and testing Your product incorporating MediaTek’s product and ensure such product meets applicable standards and any safety, security or other requirements.

The above T&C and all acts in connection with the T&C or this Document shall be governed, construed and interpreted in accordance with the laws of Taiwan, without giving effect to the principles of conflicts of law.